

Nawaloka Health Assistant

Cloud Services, Cost & Engineering Decisions

Production deployment on AWS · us-west-2 · ECS Fargate (Graviton/arm64)

AI Engineer Essentials - Weeks 15-16 · Teaching Reference

Contents

Contents.....	2
At a Glance	3
1. Summary of Services We Use	3
1.1 AWS Infrastructure	3
1.2 Third-Party AI & Data Services (SaaS)	4
2. Cost of Each Service - and Why We Use It	5
2.1 AWS Cost Breakdown.....	5
2.3 The Bill Three Ways: List Price vs Free Tier vs \$300 Credits	6
2.2 Third-Party AI Services	7
3. Cost-Saving Decisions: Two Lenses	8
3.1 Industry Mindset - the same moves a real team makes.....	8
3.2 Academic Perspective - teach the concept, don't pay to run it.....	9
4. Turning It Off - Pause vs. Destroy.....	10
4.1 What costs money while idle (and how to stop it)	10
4.2 Commands	10
Summary: What the Numbers Teach.....	12

At a Glance

What we deployed: A multi-service, horizontally-scalable AI assistant - a FastAPI app (with the React web UI baked in), an Arq background worker, and an on-demand LiveKit voice worker - running on AWS ECS Fargate behind a single Application Load Balancer, backed by managed Redis and a set of third-party AI APIs.

Headline cost: Roughly **US\$170-230 / month** for the always-on AWS footprint of one environment - which works out to about **\$5.70-7.70 / day** or **\$0.24-0.32 / hour**, plus metered third-party AI usage. The single biggest line item is networking (NAT Gateways), not compute.

This is a 1-2 day workload: students do not run this for a month. For a typical **two-day** exercise the realistic AWS spend is about **\$12-16 total - if you remember to turn things off afterwards** (see Section 4).

Headline decision: We deliberately run **one real environment** instead of three (dev / staging / production) and **teach** the full CI/CD promotion flow rather than paying to run it - cutting ~\$350-400/month while losing zero conceptual content.

READ THIS FIRST

The cloud bills by the hour, not by usage. Most of these services charge for every hour they **exist** - whether or not anyone is talking to the app. An idle, forgotten stack still costs ~\$5-7/day. The single most important habit:

tear it down (or scale to zero) the moment you finish a session. Section 4 gives the exact commands.

1. Summary of Services We Use

The stack splits into two groups: AWS infrastructure we provision and pay AWS for directly, and third-party (SaaS) AI services the application calls over the network.

1.1 AWS Infrastructure

Service	Role in the system	Configuration
ECS Fargate	Serverless container compute - runs all three services with no servers to manage	api: 1 vCPU / 2 GiB × 2 tasks worker: 0.5 vCPU / 1 GiB × 1 voice: 1 vCPU / 2 GiB × 0-1 (on-demand) Graviton / arm64
Application Load Balancer (ALB)	Public front door; distributes HTTP traffic across the 2 api replicas; health checks /health	1 ALB, HTTP listener on port 80 → target group on 8000
ElastiCache for Redis	Queue broker for the Arq worker (job hand-off between api and worker)	cache.t4g.micro, single node
NAT Gateway	Lets tasks in the VPC reach the internet (AI APIs, image pulls) without public IPs	2 × NAT GW (one per AZ, Copilot default)

Service	Role in the system	Configuration
Elastic Container Registry (ECR)	Private Docker image registry for the api / worker / voice images	3 repositories, arm64 images
SSM Parameter Store	Encrypted storage for the 21 application secrets, injected into tasks at runtime	SecureString, Copilot-managed
CloudWatch Logs	Centralised container logs and deployment troubleshooting	One log group per service
VPC / Subnets / Security Groups	Private network, public subnets, and firewall rules tying it all together	1 VPC, public placement, default networking
AWS Budgets	Cost guardrail - emails an alert at every \$20 of spend	\$20-increment alerts → ops@zuucrew.ai

1.2 Third-Party AI & Data Services (SaaS)

These are not billed by AWS; the application calls them over HTTPS. They are listed for completeness because they are part of the running system and carry their own (mostly usage-based) cost.

Service	Role	Billing shape
Supabase	Postgres + pgvector - patients, chat sessions, long-term memory	Free / flat tier
Qdrant Cloud	Vector database for the semantic (CAG) cache & RAG index	Free / flat tier
LiveKit Cloud	Real-time audio transport for the voice worker	Free dev tier / metered minutes
Deepgram	Speech-to-text (nova-3) for voice	Per-minute
ElevenLabs	Text-to-speech (eleven_turbo_v2_5) for voice	Per-character
Groq	Fast LLM (llama-3.3-70b) for the routing / voice fast path	Per-token
OpenAI / OpenRouter / Anthropic	Embeddings and chat models for the agent	Per-token
Langfuse	Tracing & latency observability	Free / flat tier
Tavily	Real-time web search tool	Per-request

2. Cost of Each Service - and Why We Use It

Figures are **approximate monthly estimates for us-west-2** assuming the always-on services run 24×7. Actual AI-API costs are usage-based and depend on traffic. Fargate prices reflect the ~20% Graviton (arm64) discount.

2.1 AWS Cost Breakdown

The **/ day** column is the monthly figure ÷ 30 - what each service actually costs for a one- or two-day exercise while it is left running. The **bracketed tag** under each price shows its **Free-Tier** status (see legend below).

Service	Est. / month (Free Tier?)	Est. / day	Why we use it (justification)
Fargate - api (2×)	~\$58 (not free)	~\$1.90	Two replicas demonstrate horizontal scaling and a shared cache/queue across instances behind the ALB. 1 vCPU / 2 GiB each is the minimum that boots the orchestrator (torch + sentence-transformers + langchain) without OOM.
Fargate - worker	~\$14 (not free)	~\$0.46	Offloads slow work (memory distillation, auto-titling) off the request path so the chat API stays fast. Smaller 0.5 vCPU / 1 GiB profile is enough.
Fargate - voice	~\$0 idle (not free)	~\$0	Scaled to 0 when not in use; ~\$0.04/hr (~\$1/day) only while a class/demo runs. Voice is bursty, so paying per-use beats a 24×7 task.
Application Load Balancer	~\$18-22 (free 12 mo*)	~\$0.65	Single public entry point, TLS termination point, and health-checked routing across replicas. Required for a real 'web service' topology.
ElastiCache (Redis)	~\$12 (free 12 mo*)	~\$0.39	Managed broker for the Arq job queue. Managed = no Redis server to patch or babysit; t4g.micro is the cheapest Graviton node.
NAT Gateway (2×)	~\$65-130 (not free)	~\$2.15-4.30	Default Copilot networking puts one NAT per AZ for outbound internet (AI APIs, image pulls). The single largest and most 'surprising' line item - and it bills 24×7 even with zero traffic. See Decision 5 and Section 4.
ECR	~\$1 (free 12 mo*)	~\$0.03	Private, region-local image storage = fast, secure pulls into Fargate. Storage is pennies per GB.
SSM Parameter Store	\$0 (always free)	\$0	Standard parameters are free. Keeps 21 secrets out of the image and out of Git; the task role is auto-granted read + decrypt.
CloudWatch Logs	~\$2-5 (always free*)	~\$0.10	Without logs you cannot debug a crash-looping task. We hit exactly this (worker Redis healthcheck) and the logs found it.
AWS Budgets	\$0 (2 free)	\$0	First two budgets are free. Non-negotiable guardrail so a runaway never produces a surprise bill.

Service	Est. / month (Free Tier?)	Est. / day	Why we use it (justification)
Approx. AWS total	~\$170-230	~\$5.70-7.70	One always-on environment. Compute (~\$72) + ALB + Redis are predictable; NAT is the swing factor. A 2-day run ≈ \$12-16 if torn down afterward.

Legend: “not free” = no Free-Tier allowance, you pay from dollar one. “free 12 mo” = covered by the 12-month new-account Free Tier. “always free” = permanently free within the limit. * = **within Free-Tier limits** (e.g. one small instance / first 750 hours / first few GB); usage beyond the limit is billed normally.

Reading the table: notice that **networking (NAT) rivals all of compute combined**. This is the classic cloud-cost lesson - the bill is rarely where beginners expect it. Compute is cheap and elastic; the steady-state ‘plumbing’ (load balancer + NAT + managed data services) is what you actually pay for. Note too that the **two un-free items are exactly Fargate and NAT** - so even with the Free Tier, those are what you (or your credits) pay for.

2.3 The Bill Three Ways: List Price vs Free Tier vs \$300 Credits

The same deployment costs wildly different amounts depending on what you have access to. We compute it **without** hiding the real list price, then show what the Free Tier and a typical **\$300 new-account credit** actually leave you paying.

Scenario	You actually pay	What is happening
A. Full list price (no Free Tier, no credits)	~\$170-230 / mo ~\$5.70-7.70 / day ~\$12-16 / 2-day	The true, undiscounted cost. This is what an existing AWS account with the Free Tier already expired would pay. Always quote this number so nobody is surprised later.
B. With 12-month Free Tier (new account, year 1)	~\$140-205 / mo ~\$4.60-6.80 / day	ALB, ElastiCache, ECR, SSM, CloudWatch drop toward \$0 within their limits. What REMAINS is the two un-free items: Fargate (~\$72) + NAT (~\$65-130). The Free Tier helps, but does not make this deployment free.
C. With Free Tier + \$300 credits	\$0 out of pocket (until credits run out)	Credits are real dollars that pay ANY charge, so they absorb the Fargate + NAT remainder from row B. You pay nothing until the \$300 (or the credit window) is exhausted.

STRETCHING THE \$300

How far does \$300 go? In scenario C, credits only need to cover the un-free remainder (~\$140-205 / mo ≈ ~\$4.60-6.80 / day).

Left running 24×7: $\$300 \div \sim\$5.70/\text{day} \approx 45\text{-}65$ days before the credits are gone.

Run only during class, then torn down (Section 4): a 2-day session burns ~\$9-14 of credit, so $\$300 \approx 20\text{-}30$ separate sessions.

The lesson: credits make it feel free, but they are a finite tank - the same shut-down discipline decides whether \$300 lasts six weeks or six months.

READ THE FINE PRINT

Caveats on credits. (1) Credits require a **new** account. (2) They **expire** - the 2025 free plan caps near \$200 over ~6 months; older promos gave \$300 over 12 months. (3) Free-Tier 12-month allowances also end after one year. Quote scenario A as the 'true' cost so students plan for the day the free help runs out.

2.2 Third-Party AI Services

For this teaching deployment every vendor sits on a **free or low flat tier**, so the marginal cost during class is effectively zero. In a real product these become the **dominant** cost as traffic grows - LLM tokens, STT minutes and TTS characters scale linearly with users, while the AWS base stays roughly flat. That inversion (infra flat, AI usage linear) is itself a key thing to teach.

3. Cost-Saving Decisions: Two Lenses

Every choice below was made twice over - once asking “what would a real engineering team do?” (industry lens) and once asking “what teaches the concept without wasting students’ money?” (academic lens). The strongest decisions satisfy both.

3.1 Industry Mindset - the same moves a real team makes

Decision 1 - ECS Fargate instead of self-managed servers (or Kubernetes).

- **What:** Run containers on Fargate - no EC2 instances, no patching, no cluster to operate.
- **Why (industry):** A small team ships faster and pays only for the vCPU/GiB a task actually uses. No idle EC2 fleet, no Kubernetes control-plane overhead or specialist ops cost. This is the default for most modern small-to-mid production workloads.
- **Saving:** Eliminates a full-time ops burden and the cost of over-provisioned, always-on instances.

Decision 2 - Graviton (arm64) for all compute.

- **What:** Build arm64 images (native on the Apple-Silicon dev machine) and run them on Graviton Fargate.
- **Why (industry):** ~20% cheaper per vCPU-hour for identical performance on this workload, with no code changes. ‘Free’ money that real teams take.
- **Saving:** ~20% off every compute line (~\$15+/month here, far more at scale).

Decision 3 - On-demand voice (scale to zero).

- **What:** The voice worker runs at **0 tasks** by default and is scaled to 1 only for a live demo or call.
- **Why (industry):** Voice traffic is bursty and expensive to keep warm. Pay-per-use for spiky workloads is standard practice; a 24x7 voice task would cost ~\$28/month to sit idle.
- **Saving:** ~\$28/month → a few cents per session.

Decision 4 - One container serves both UI and API.

- **What:** The React web app is built and baked into the api image; FastAPI serves it at / and routes /api/* to the backend behind the same ALB.
- **Why (industry):** One hostname, no CORS, no second deployment, and no extra CDN/compute to pay for. A common, pragmatic ‘monolith-serves-SPA’ pattern for small products.
- **Saving:** Avoids a separate static-hosting service and its operational surface; \$0 extra infra.

Decision 5 - Managed services over self-hosted (Redis, Postgres, vectors).

- **What:** ElastiCache, Supabase and Qdrant Cloud instead of running our own Redis/Postgres/vector servers.
- **Why (industry):** Buying back reliability and patching for a few dollars a month is almost always cheaper than an engineer’s time. Teams self-host only when scale or compliance forces it.
- **Honest trade-off:** The **NAT Gateway (~\$65-130/mo)** is the price of the simple ‘default’ network. A real cost-optimised team would replace NAT with **VPC endpoints** or public-subnet placement to cut most of it - we kept the default deliberately to keep the teaching setup simple, and flagged it as the #1 thing to optimise next.

3.2 Academic Perspective - teach the concept, don't pay to run it

Decision 6 - One real environment instead of three (dev / staging / production).

- **What:** We provision a single live AWS environment. A true dev→staging→prod setup means three copies of everything - three VPCs, ALBs, NATs, Redis clusters and task fleets.
- **Why (academic):** Three environments cost roughly **\$500-600/month** and teach **nothing new** - they are the same stack three times. The transferable skill is the **promotion workflow**, not the triplicated infrastructure.
- **Saving:** ~\$350-400/month avoided, with zero loss of learning outcomes.

Decision 7 - Teach the CI/CD promotion flow; don't run all three stages live.

- **What:** Students see the real branch model (dev / staging / production), CI gates, and the **manual approval before production** - but staging and production deploy steps are demonstrated in plan / dry-run mode against the one real environment.
- **Why (academic):** The GitHub Actions workflow file and the green check-marks carry 100% of the concept. The **only** real difference for production is one config value pointing at another environment - "adding real staging/prod is one command, not a new concept."
- **Saving:** \$0 extra (GitHub Actions is free for the repo); avoids the multi-environment infra cost entirely while still showing the artifact.

Decision 8 - Free / flat tiers for every AI vendor.

- **What:** Reuse existing free or low-tier accounts (Supabase, Qdrant, LiveKit, Langfuse, etc.) rather than provisioning paid plans.
- **Why (academic):** Students can reproduce the entire stack for ~\$0 of vendor cost during the course, then upgrade only the specific service that hits a limit in their own project.

Decision 9 - A hard cost guardrail from day one.

- **What:** AWS Budgets emails an alert at every \$20 of spend.
- **Why (both lenses):** Teaches financial responsibility and protects a student from a forgotten resource turning into a painful bill - the cloud-cost mistake everyone makes once.

4. Turning It Off - Pause vs. Destroy

The golden rule: almost every AWS service here is billed **per hour that it exists**, not per request. Leaving the stack up overnight 'just in case' costs the same as using it hard. When you finish a session, stop paying.

There are two ways to stop the meter. Pick based on how soon you will need it back.

OPTION A - LIGHT PAUSE

Strategy A - Pause (same day / next day). Scale the Fargate tasks to **0**. Saves the compute (~\$2.40/day) and is **instant** to reverse. The ALB, NAT and Redis keep running, so you still pay ~\$3-5/day for them.

OPTION B - FULL TEARDOWN

Strategy B - Destroy (done for a while). Tear the whole environment down → **~\$0/day**. Your code, images and secrets are safe in Git/ECR/SSM; only the running infrastructure is removed. Rebuilding takes ~15-20 minutes.

4.1 What costs money while idle (and how to stop it)

Service	Bills while idle?	How to stop paying	How to bring it back
Fargate tasks (api / worker / voice)	Only while running	Scale desired-count to 0 - no charge at 0 tasks. (voice is already 0.)	Scale desired-count back to 2 / 1 / 1. Tasks are healthy in ~2 min.
Application Load Balancer	Yes - hourly	Cannot be 'paused'. Removed only by deleting the service/environment.	Recreated automatically on the next env/service deploy.
NAT Gateway (2x)	Yes - hourly (biggest)	Cannot be 'paused'. Removed only by deleting the environment (it owns the VPC).	Recreated when you re-init/deploy the environment.
ElastiCache (Redis)	Yes - hourly	Delete the cache cluster. It is only a queue broker - no important data is lost.	Recreate via the redis deploy script; re-wire REDIS_URL.
ECR / SSM / CloudWatch / Budgets	≈ Free	Leave them. Pennies or free; deleting saves nothing meaningful and loses your images/secrets.	n/a - they persist.

4.2 Commands

Option A - scale Fargate to 0 (pause), then back up:

```
CL=nawaloka-dev-Cluster-KpIKFXAtIZpt    # your cluster name
# pause
for s in api worker voice; do \
  SVC=$(aws ecs list-services --cluster $CL \
    --query "serviceArns[?contains(@,'$s')]" --output text); \
```

```
aws ecs update-service --cluster $CL --service $SVC --desired-count 0; done
# resume (api→2, worker→1, voice→1)
aws ecs update-service --cluster $CL --service <api-svc> --desired-count 2
aws ecs update-service --cluster $CL --service <worker-svc> --desired-count 1
aws ecs update-service --cluster $CL --service <voice-svc> --desired-count 1
```

Option B - full teardown (stops ~100% of the bill), then rebuild:

```
# tear EVERYTHING down - services, ALB, NAT, VPC, cluster
copilot app delete          # one command removes the whole app
aws cloudformation delete-stack --stack-name nawaloka-redis-dev # Redis

# rebuild later (~15-20 min)
copilot env init  && copilot env deploy --name dev
./scripts/aws/deploy_redis.sh          # recreate + wire Redis
./scripts/aws/build_push_images.sh     # push images
copilot svc deploy --name api --env dev # repeat for worker, voice
```

DON'T FORGET

The cheapest habit of all: if you only paused (Option A) but won't touch it for days, come back and do Option B - an idle ALB + NAT + Redis still quietly bills ~\$3-5/day. And always confirm in the **AWS Billing console** that the meter has actually stopped.

Summary: What the Numbers Teach

Lens	The lesson
Industry	Pay for outcomes, not servers: Fargate + Graviton + managed services + scale-to-zero. Watch the plumbing - NAT/ALB, not compute, dominate a small bill.
Academic	Spend money only where it changes the learning. One environment + a taught (not run) CI/CD pipeline delivers the full concept for ~\$350-400/month less.
The inversion	AWS base cost is roughly flat; AI-API usage scales with users. At scale, the model bill - not the infrastructure - becomes the thing to optimise.

Bottom line: the architecture is genuinely production-shaped and horizontally scalable; the cost is controlled by engineering choices (industry) and scoping choices (academic) - not by cutting corners on the design.